

SIA TP N°3 - Perceptrón Simple y Multicapa

Lautaro BONSEÑOR | Nicolás RIVAS | Juan Ignacio CAUSSE
Javier Emmanuel LIU | Augusto Andrés LANARI



Implementación

Implementación

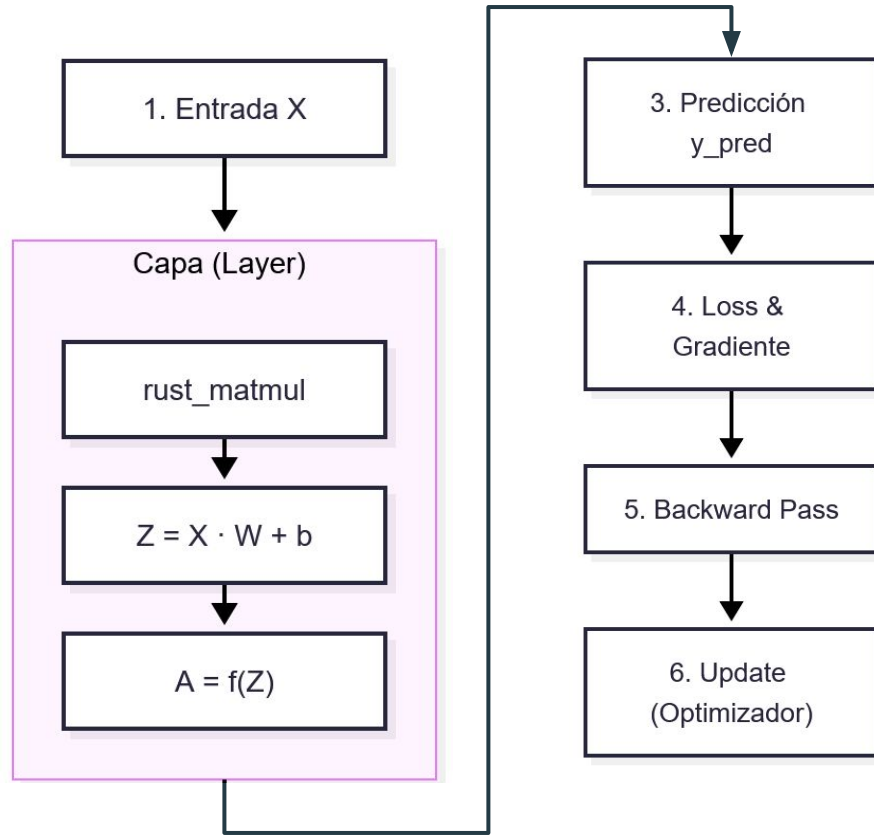
Multi-layer Perceptron

Aceleración y Operaciones Matriciales en Rust

Flexibilidad Modular y Orientada a Objetos

Gestión de Experimentos y Persistencia

Arquitectura



Multiplicación Matricial en Rust



```
0.03s      def feed_forward(self, x):
3.41s      self.input = x
0.3s      self.z = x @ self.W + self.b
          return self.activation_function.compute(self.z)
```

```
0.97s      def backward(self, grad_output):
4.07s      grad_z = grad_output * self.activation_function.gradient(self.z)
0.38s      grad_W = self.input.T @ grad_z
          grad_b = np.sum(grad_z, axis=0)

3.36s      grad_input = grad_z @ self.W.T
          return grad_input, grad_W, grad_b
```

real	02m	46.097s
user	45m	58.156s
sys	02m	53.849s

```
0.28s      def feed_forward(self, x: NDArray[np.float64]) -> NDArray[np.float64]:
2.43s      self.input = x
0.3s      self.z = rust_matmul.matmul(x, self.W) + self.b
          return self.activation_function.compute(self.z)
```

```
0.95s      def backward(
self, grad_output: NDArray[np.float64]
) -> tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64]]:
2.16s      grad_z = grad_output * self.activation_function.gradient(self.z)
0.3s      grad_W = rust_matmul.matmul(self.input.T, grad_z)
          grad_b = np.sum(grad_z, axis=0)

2.3s      grad_input = rust_matmul.matmul(grad_z, self.W.T)
          return grad_input, grad_W, grad_b
```

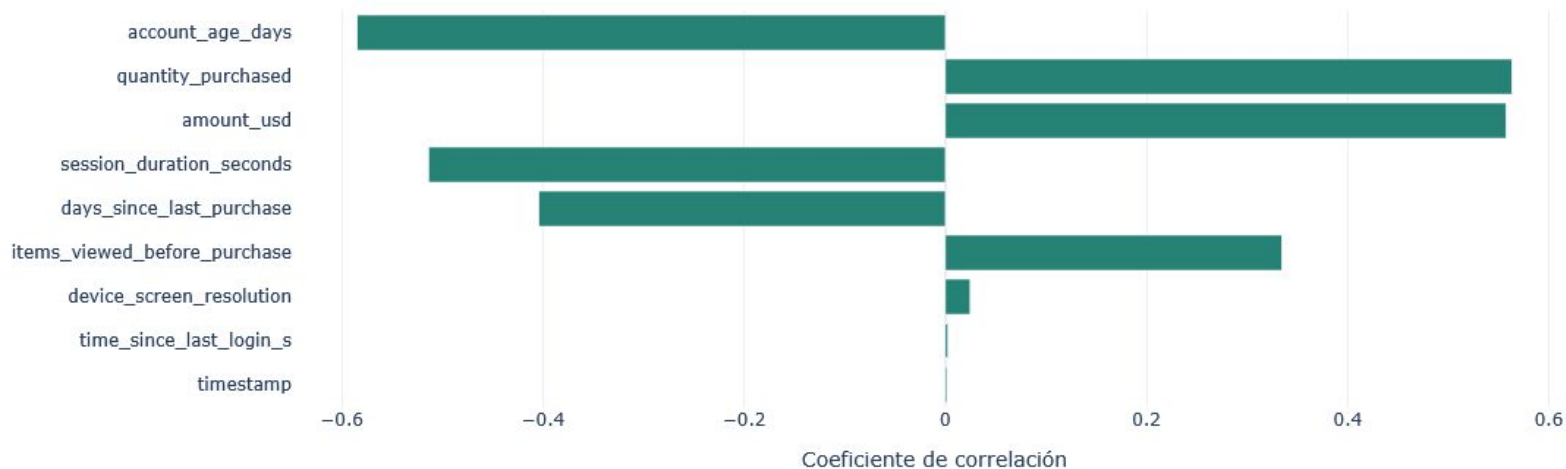
real	02m	08.412s
user	10m	41.999s
sys	04m	46.749s



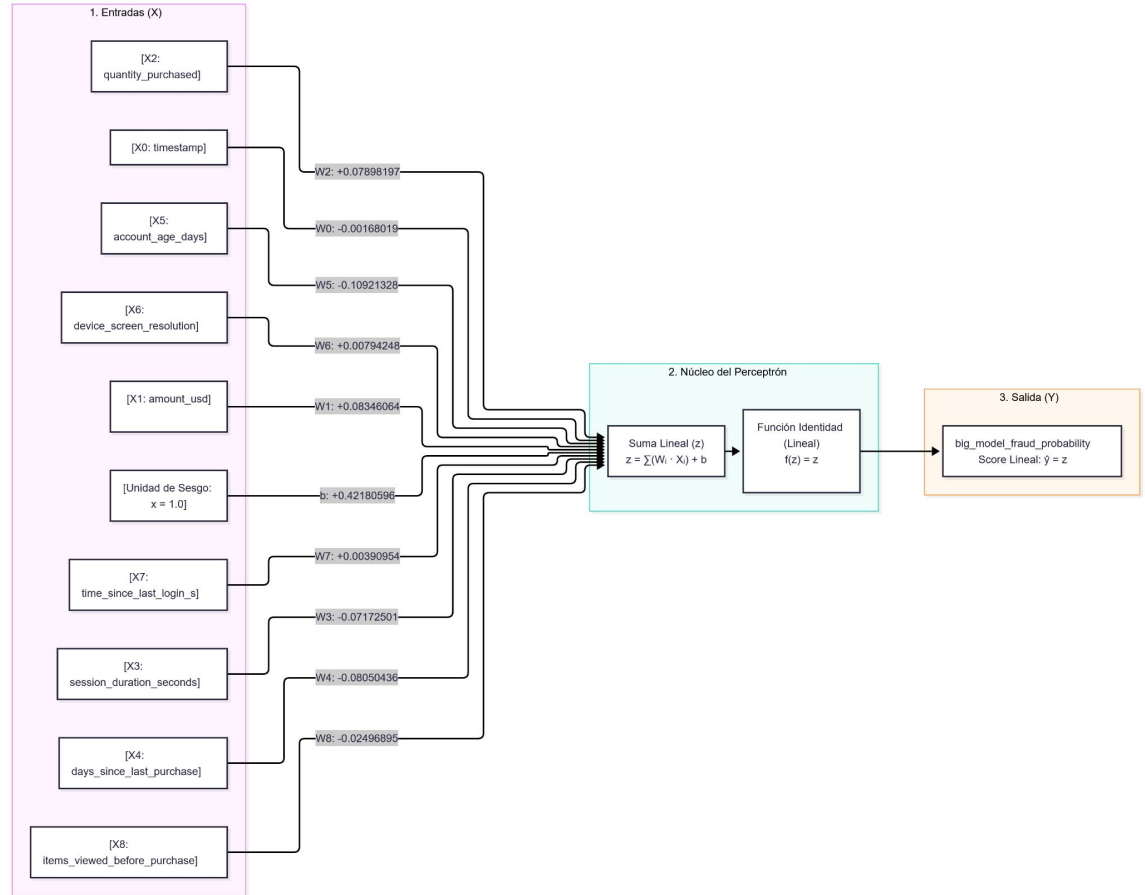
TinyModel

Exploración del Dataset

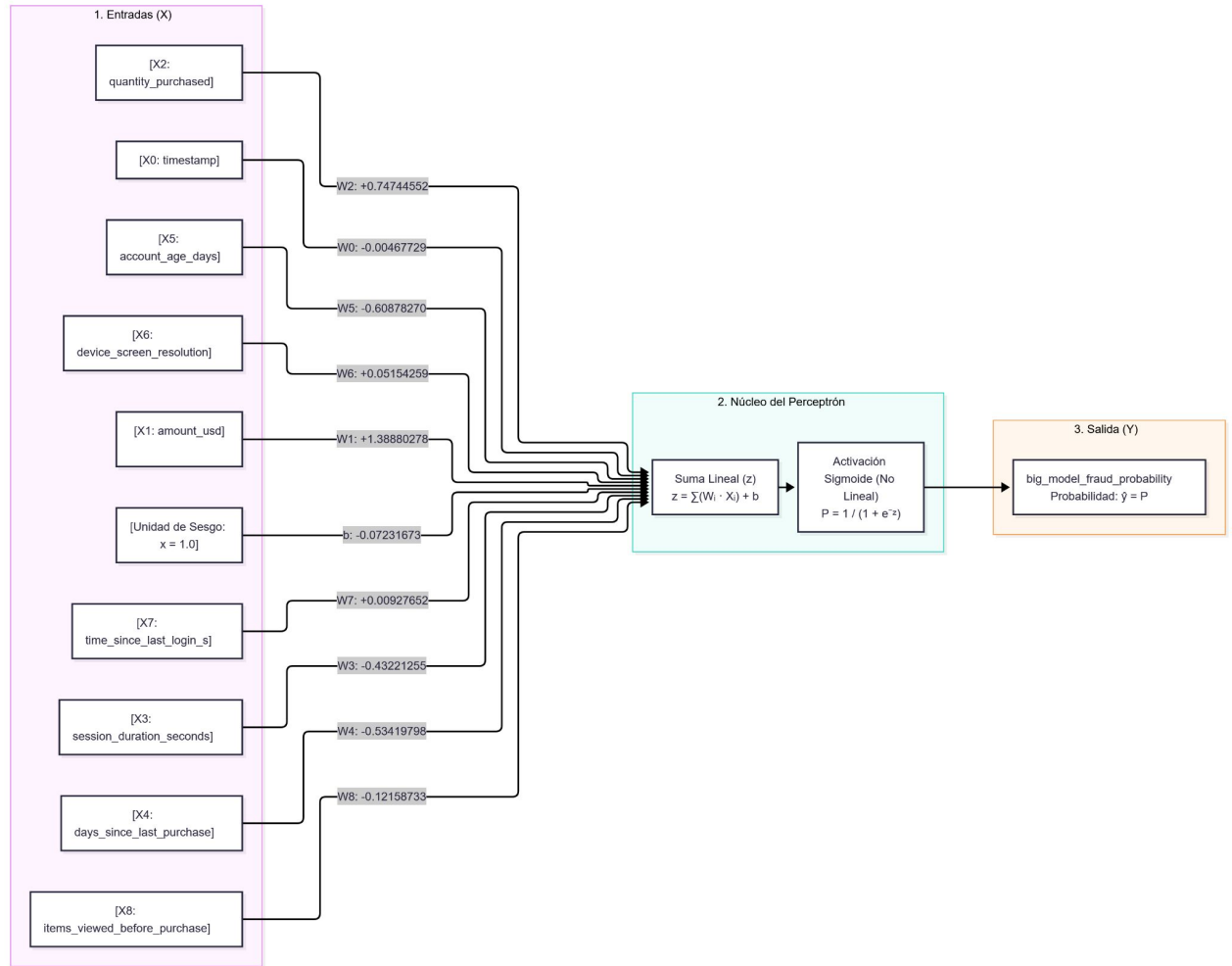
Correlación de Pearson de cada variable con el objetivo



Perceptrón Lineal

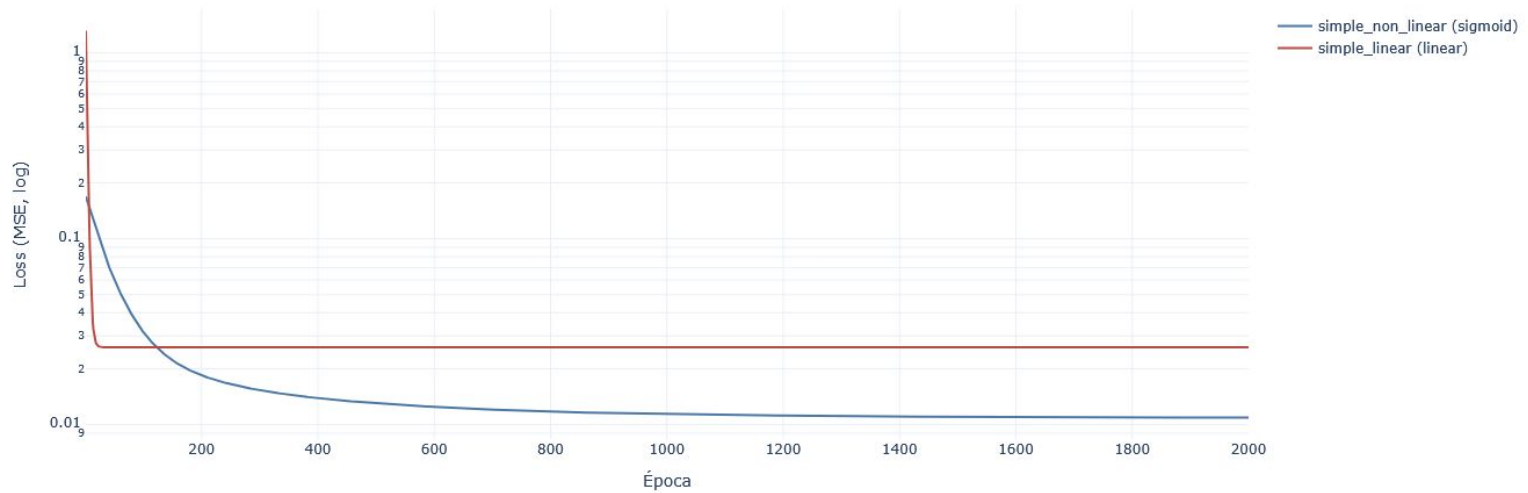


Perceptrón No Lineal



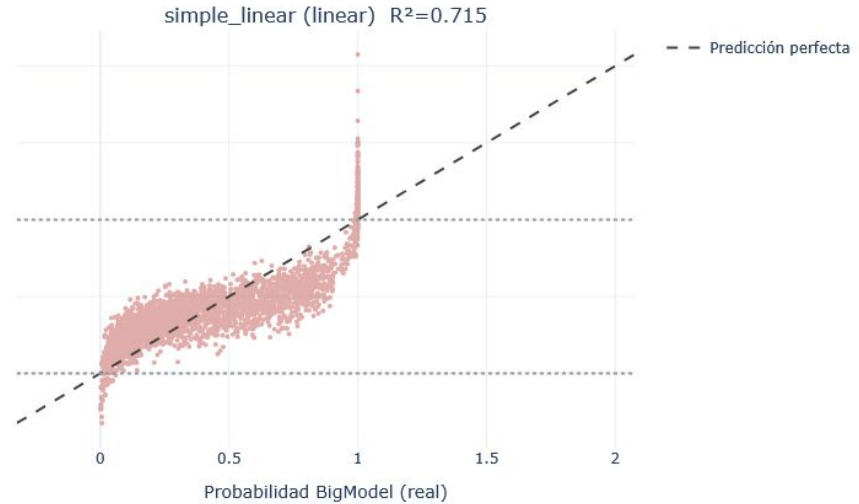
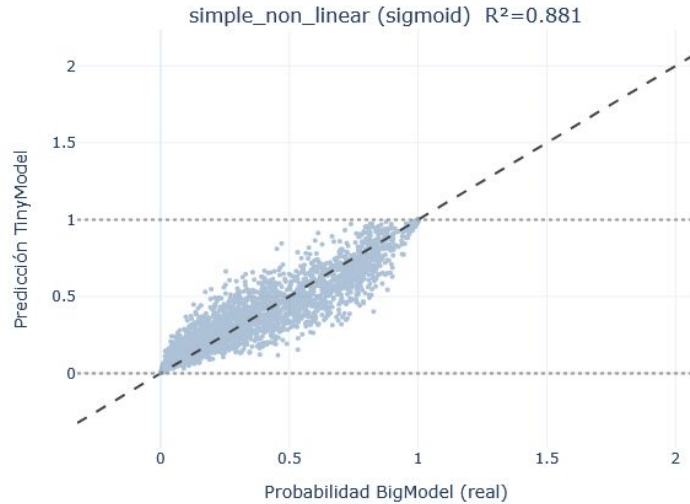
Lineal vs No Lineal - Loss

Loss por época, todas las muestras (7,500), escala log



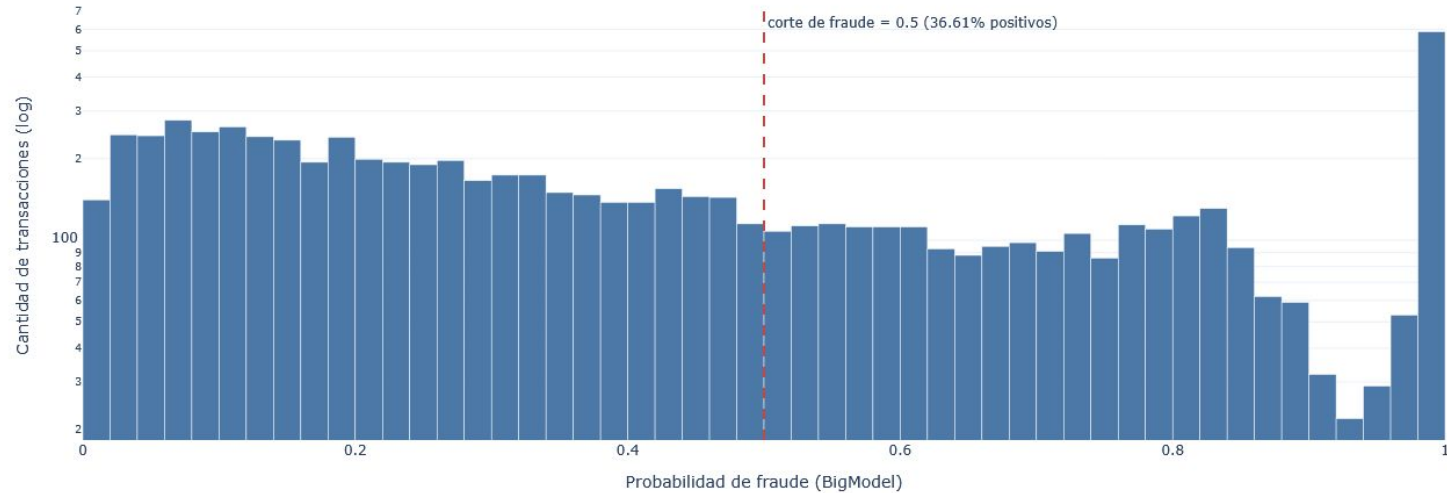
Lineal vs No Lineal - Predicción vs Actual

Predicho vs real (todas las muestras; líneas punteadas = 0 y 1)



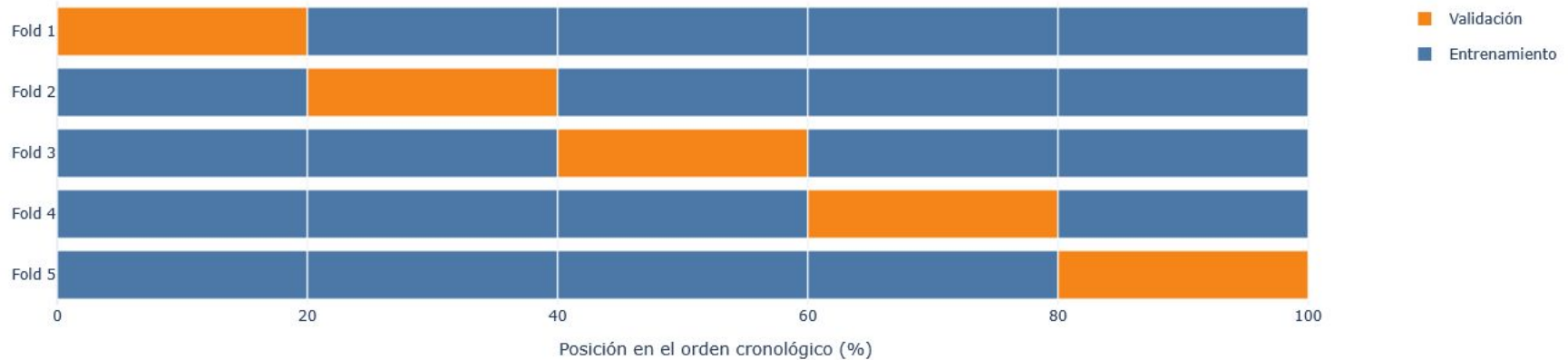
Diseño del estudio de generalización

Distribución de la probabilidad de BigModel (7,500 filas)



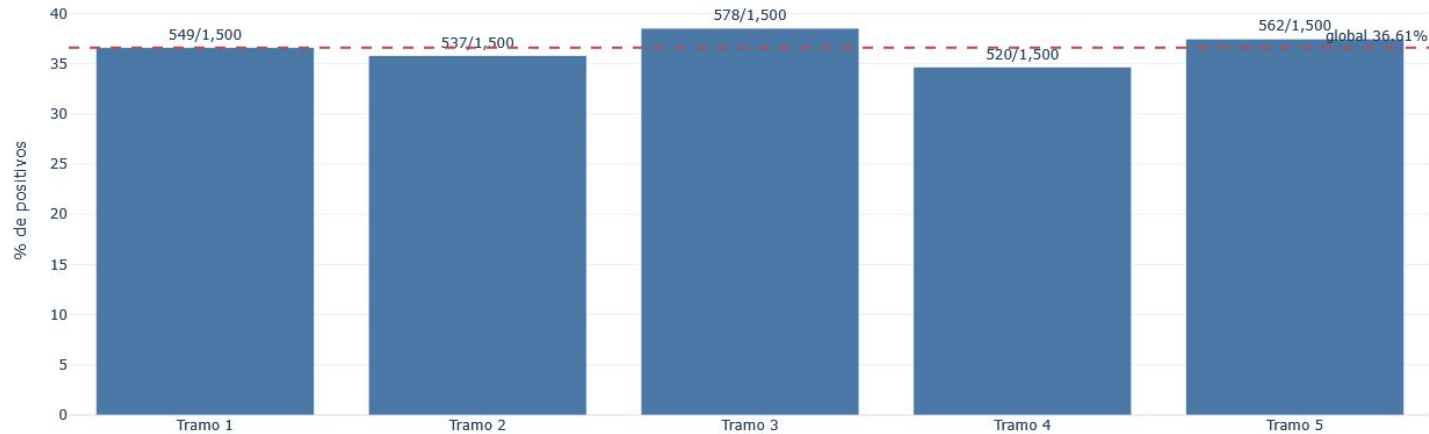
Diseño del estudio de generalización

Esquema de folds sobre el orden cronológico: k-fold (entrena también con tramos futuros)



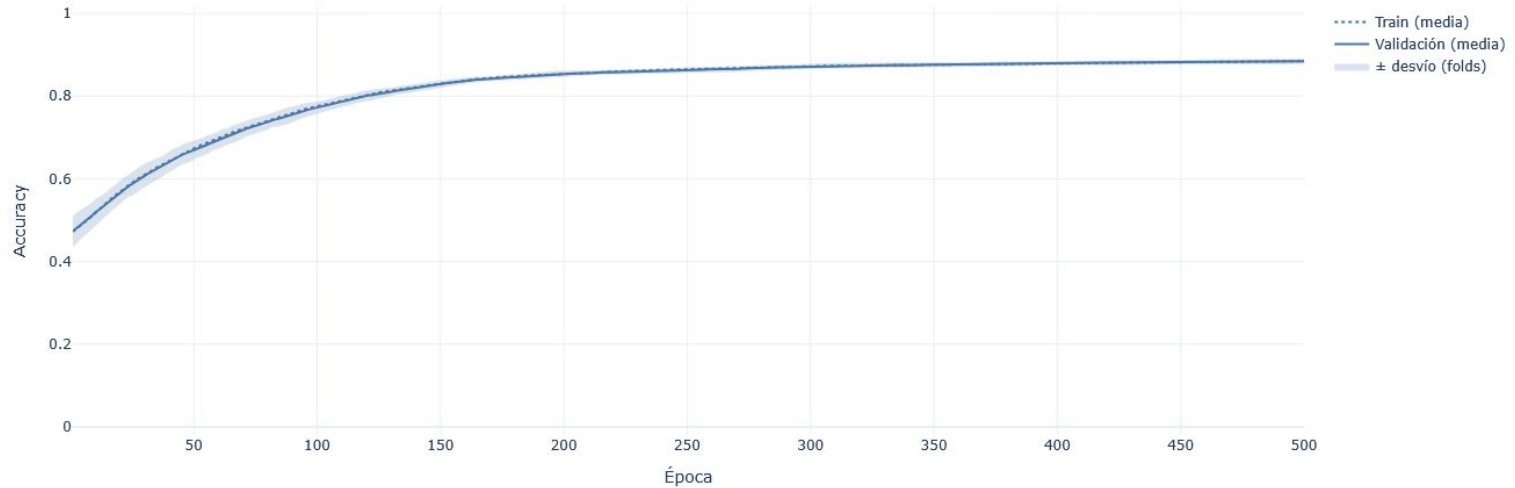
Diseño del estudio de generalización

% de fraude (prob. ≥ 0.5) por tramo cronológico



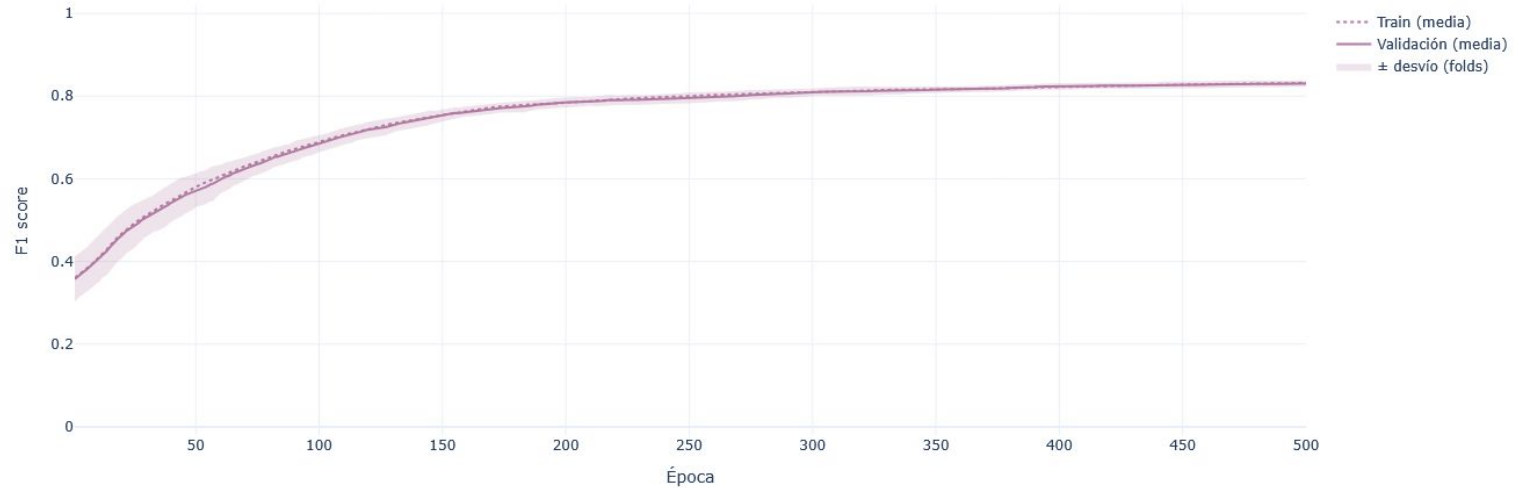
Resultados por época - Accuracy

Accuracy por época (primeras 500 épocas; media de 5 folds, umbral 0.5, corte 0.5)



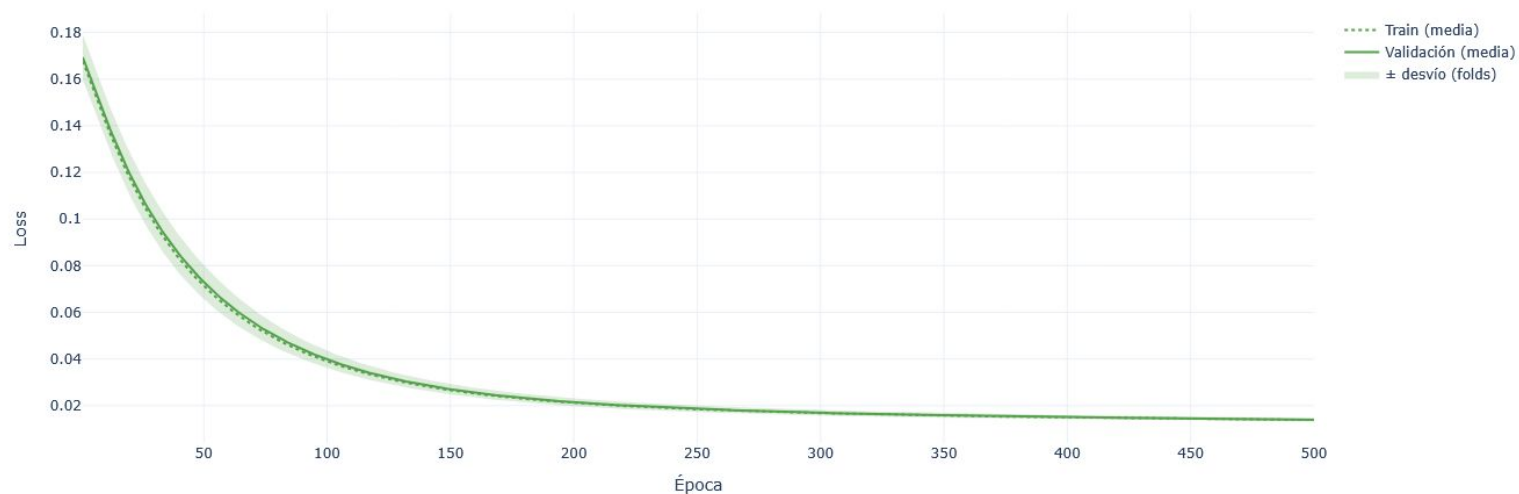
Resultados por época - F1 score

F1 score por época (primeras 500 épocas; media de 5 folds, umbral 0.5, corte 0.5)



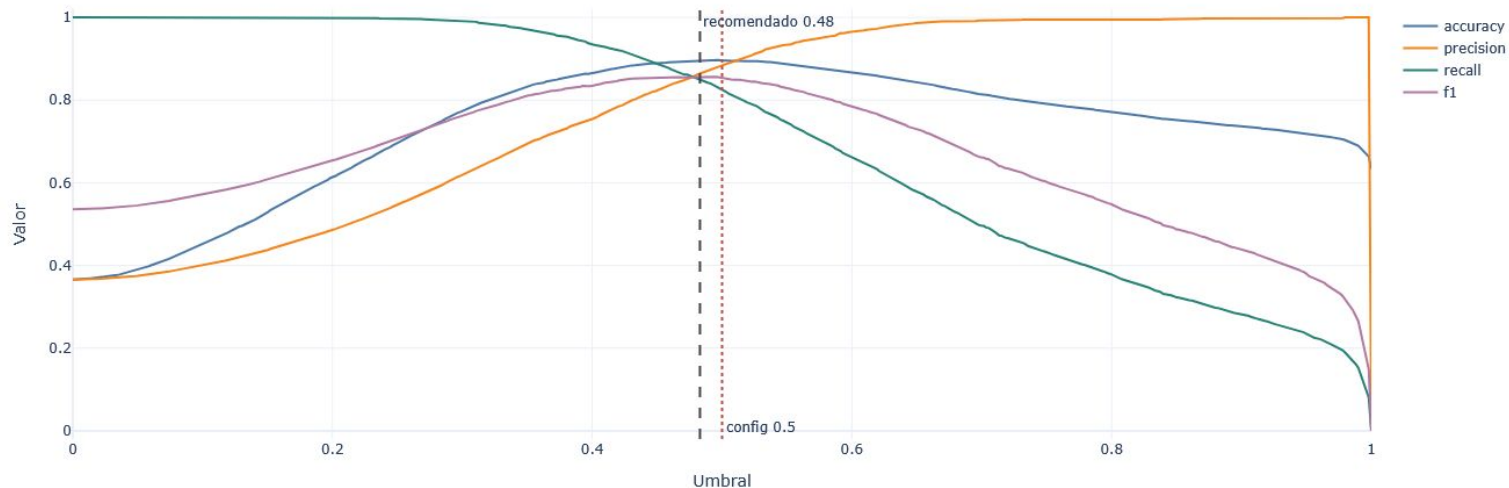
Resultados por época - Loss

Loss por época (primeras 500 épocas; media de 5 folds, umbral 0.5, corte 0.5)

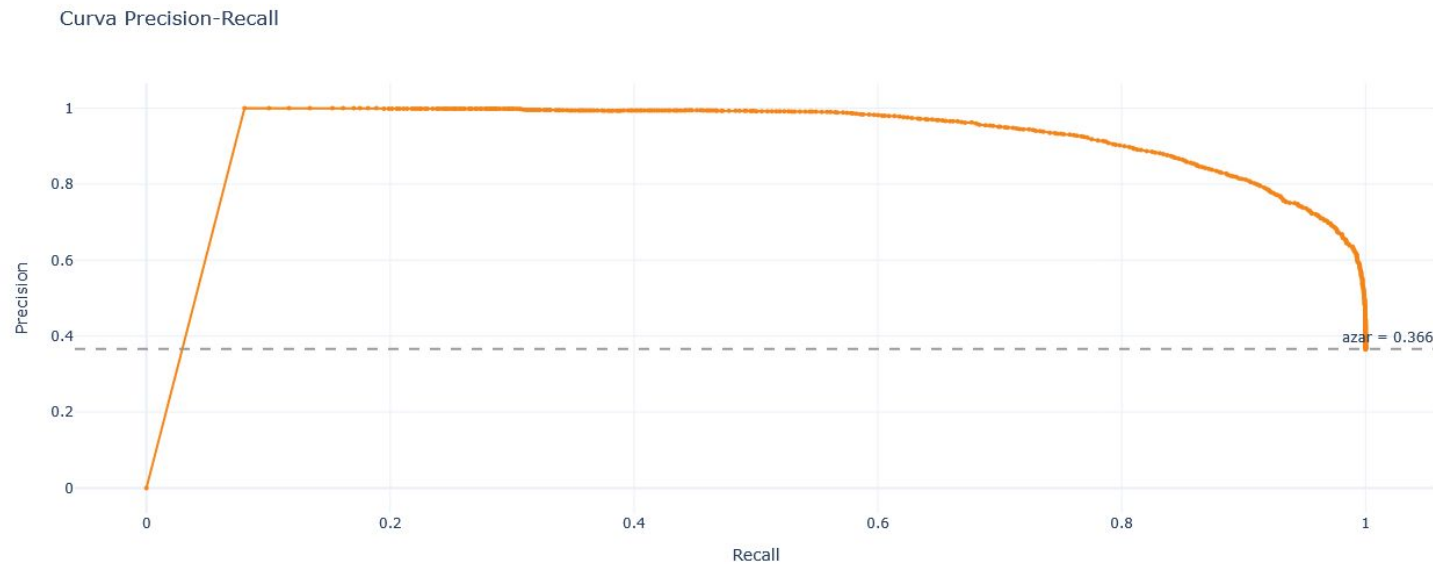


Análisis por umbral - Umbral vs Métricas

Accuracy / Precision / Recall / F1 vs umbral (predicciones fuera de muestra)

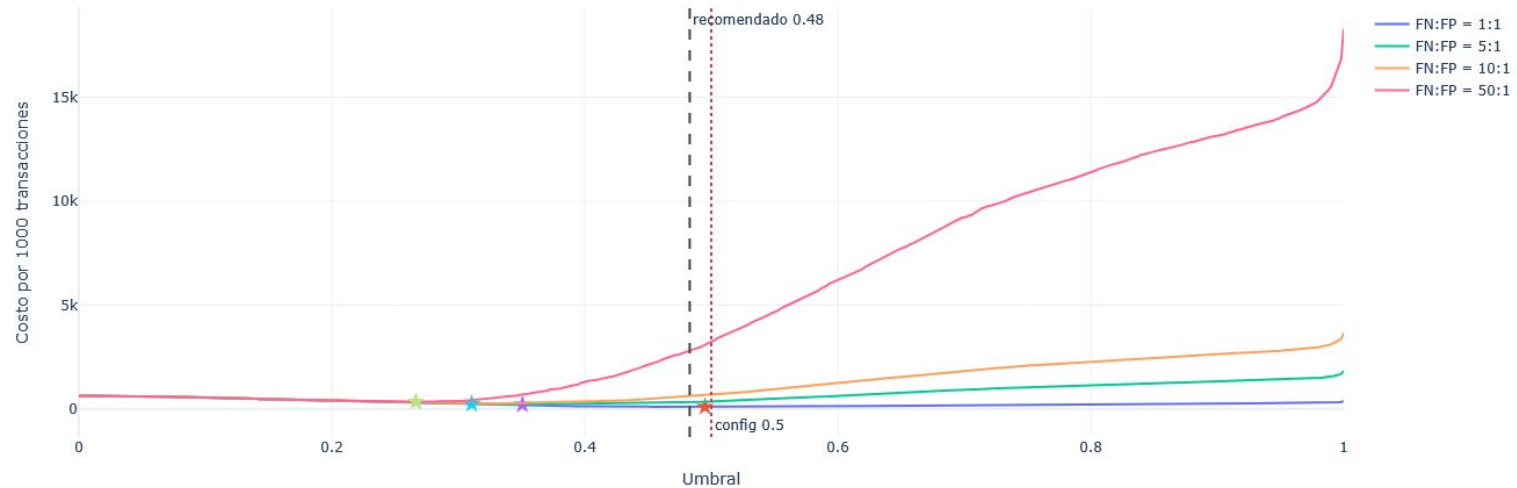


Análisis por umbral - Precisión vs Recall



Análisis por umbral - Costo vs Umbral

Costo esperado por 1000 transacciones vs umbral (unidad = costo de un falso positivo)

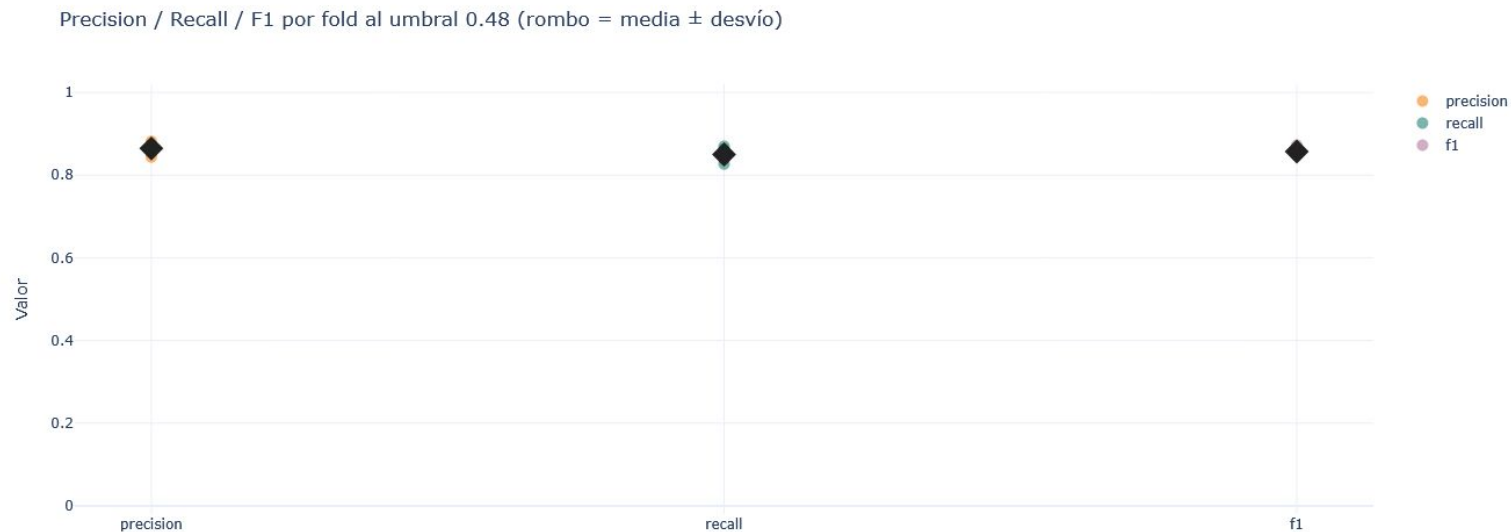


Modelo final y recomendación de umbral.

Matriz de confusión al umbral 0.48 (fuera de muestra, máximo F1)

Real: no fraude	4,388 92.3% de la fila	366 7.7% de la fila
Real: fraude	413 15.0% de la fila	2,333 85.0% de la fila
	Predicho: no fraude	Predicho: fraude

¿Qué tan sólidas son las conclusiones?

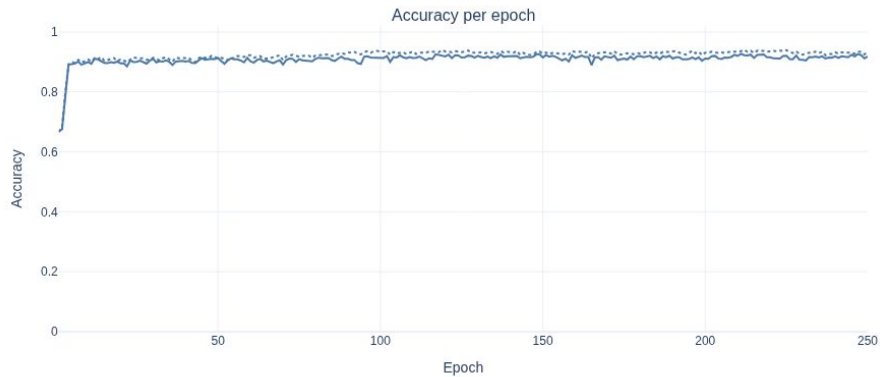
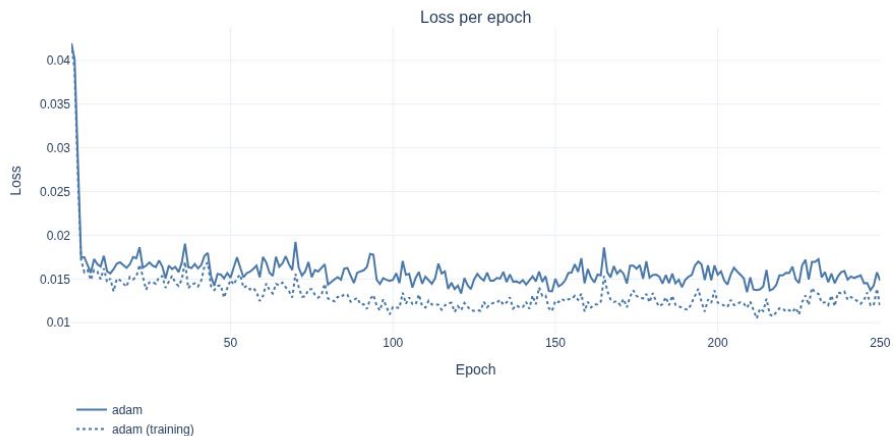




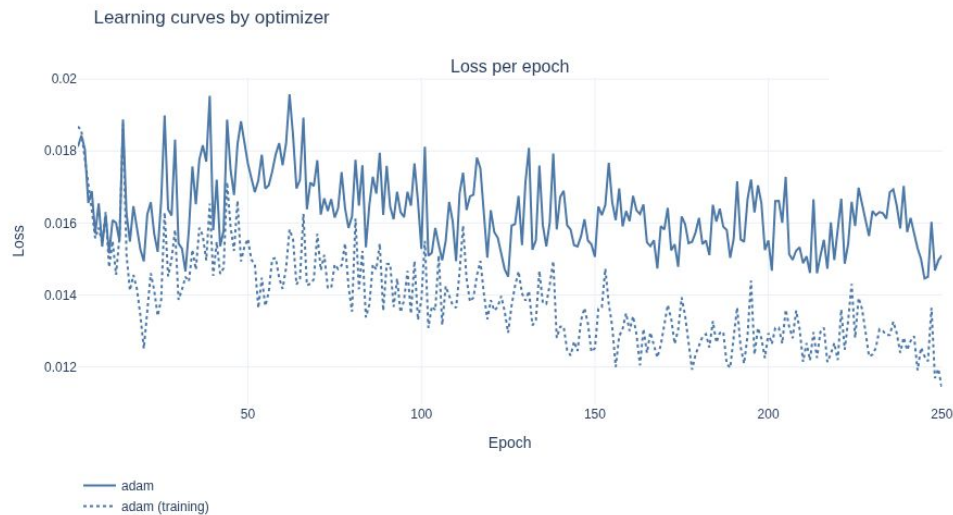
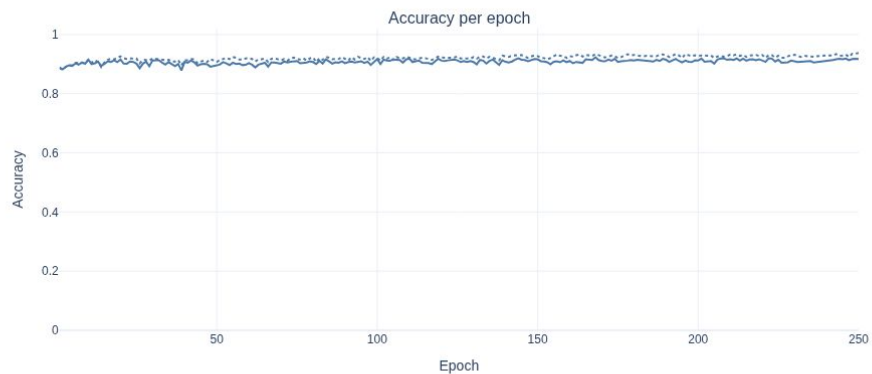
Dígitos con MLP

Inicialización pesos random

Learning curves by optimizer



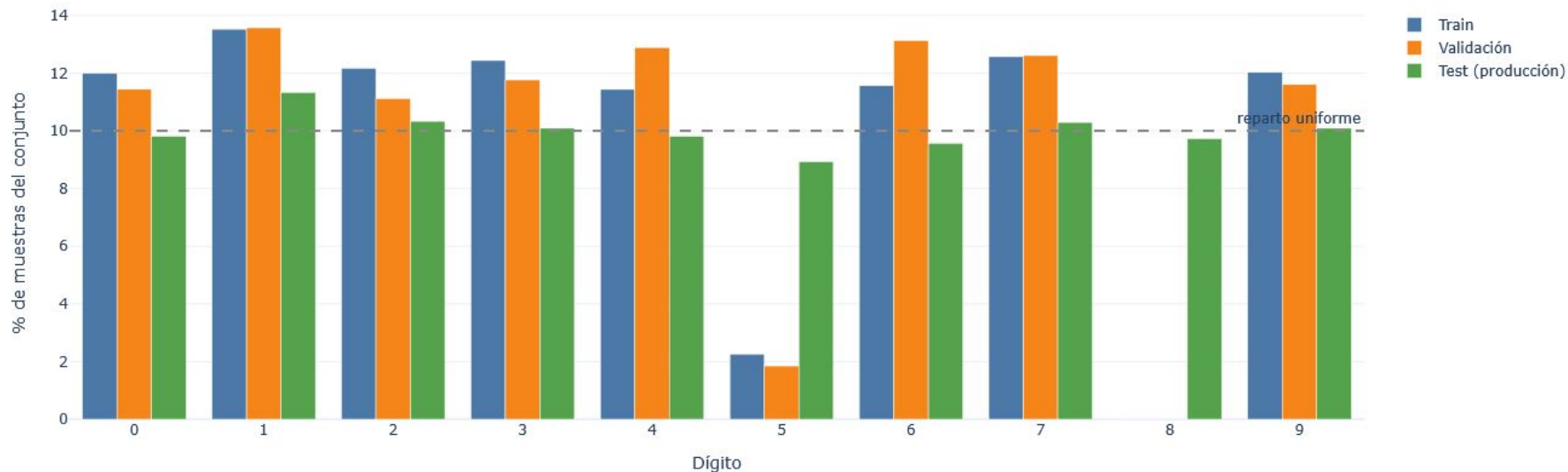
Inicialización de Kaiming He



Protocolo de evaluación

0 1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9 0
2 3 4 5 6 7 8

Balance de clases por conjunto



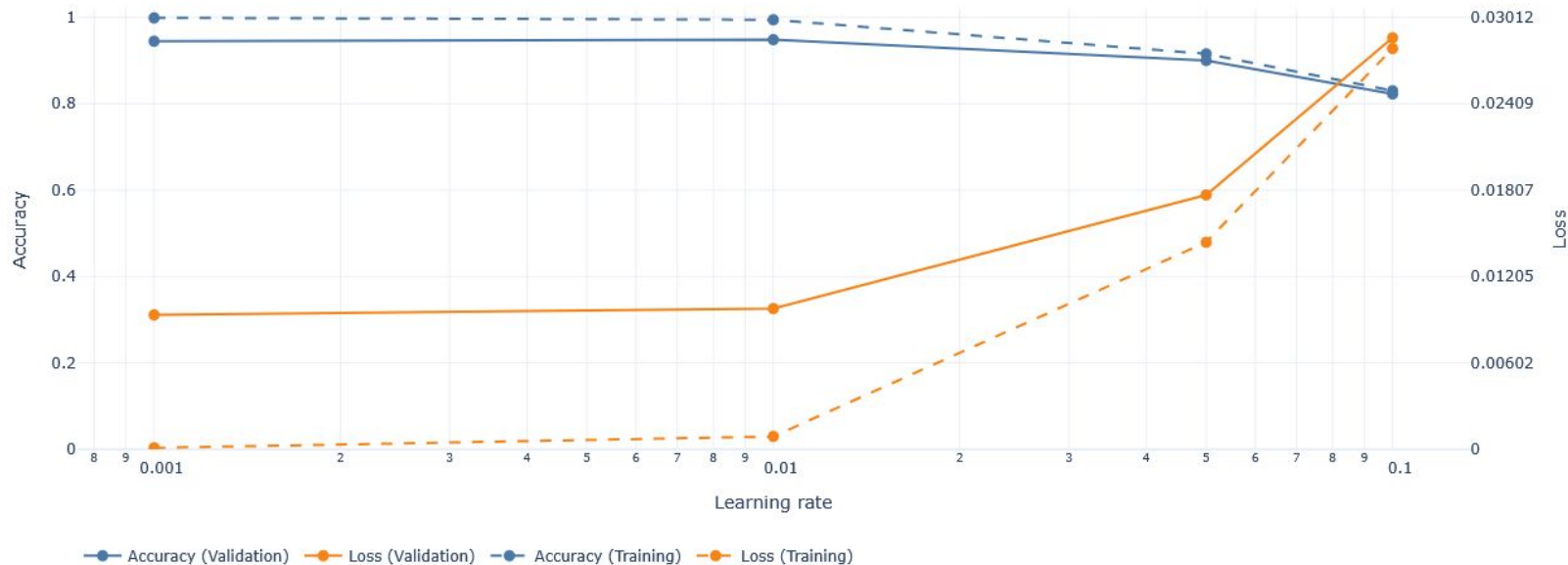
Protocolo de evaluación

Protocolo de evaluación (split 80% / 20%; el test no se usa para decidir nada)

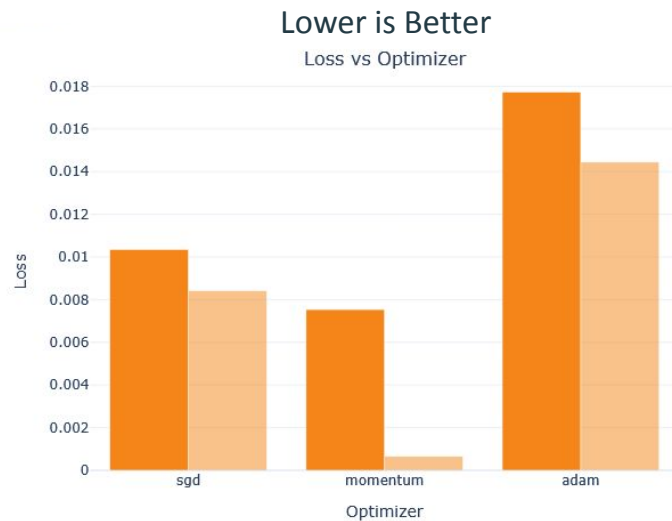
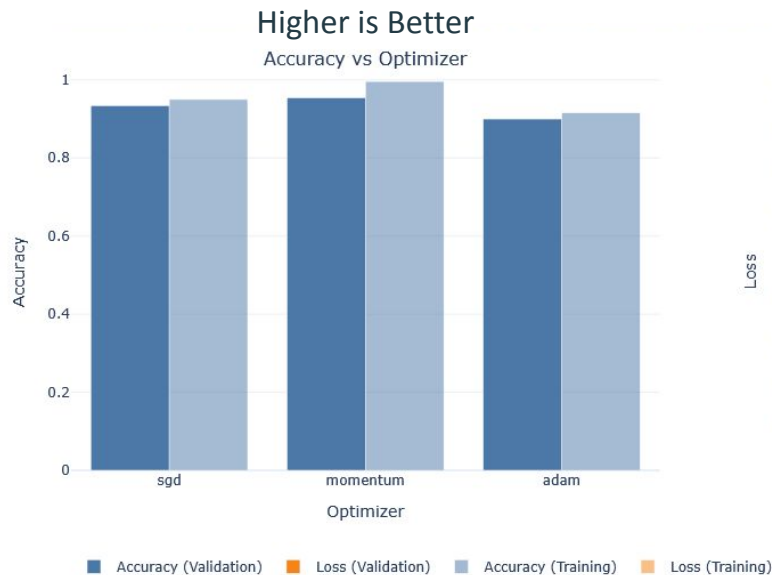


Barridos de hiperparámetros - Learning Rate

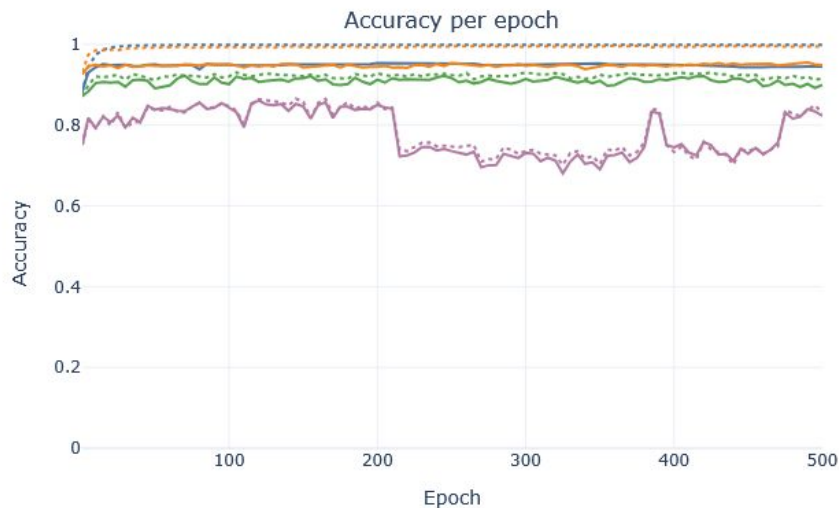
Accuracy and Loss vs Learning Rate (adam; 500 epochs, batch 32, 1 seed(s))



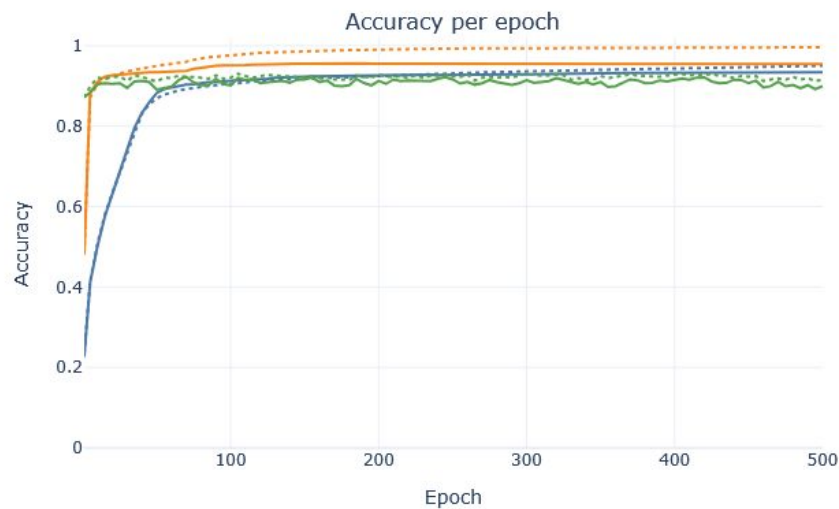
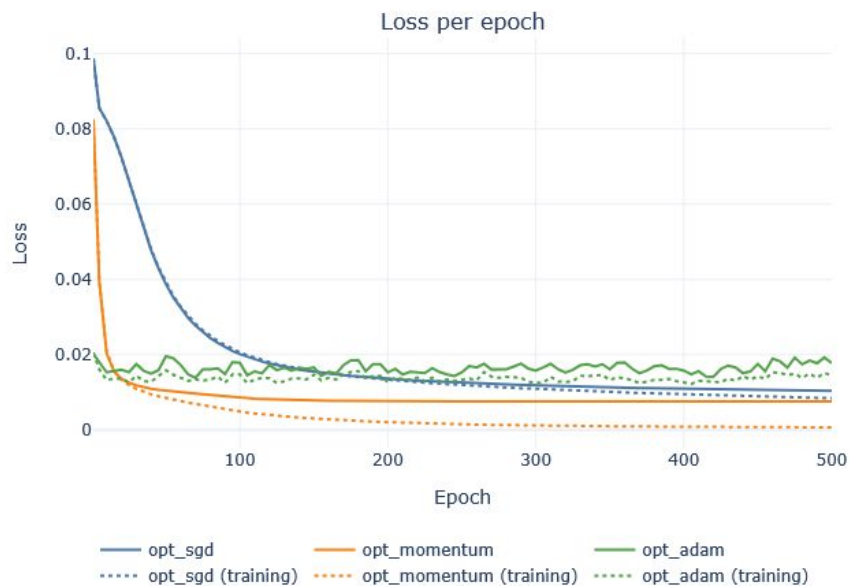
Barridos de hiperparámetros - Optimizer



Interacción LR × optimizador y curvas de aprendizaje



Interacción LR × optimizador y curvas de aprendizaje



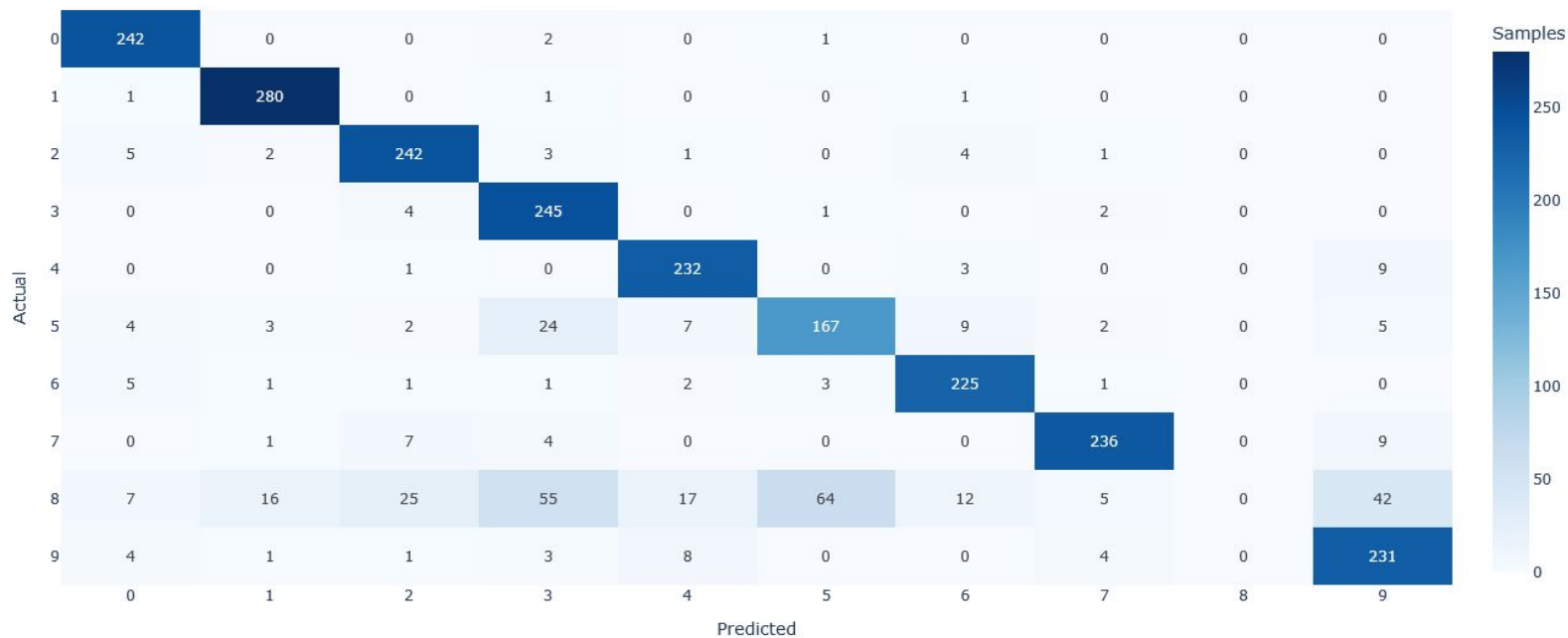
Mejor Modelo

OPTIMIZADOR | LR | HIDDEN | OUTPUT

Momentum | 0.1 | ReLU | Sigmoid

Mejor modelo

Confusion Matrix (test) — best model: momentum / lr=0.1 [grid], test_acc=0.8410

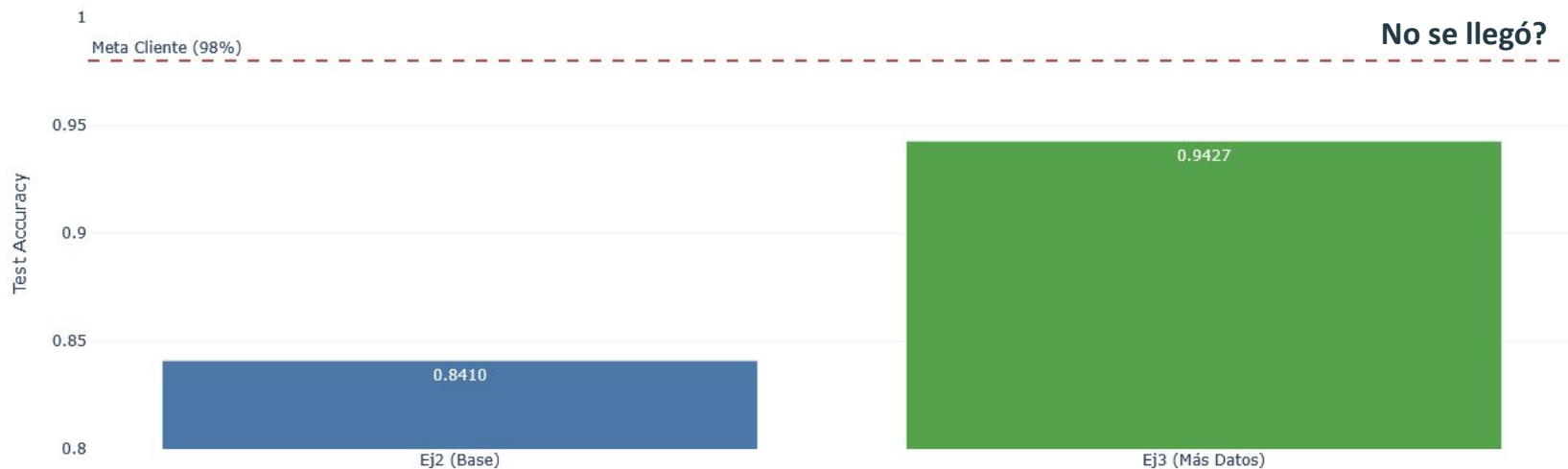




Más dígitos...

Resultados con más datos

Mejora de Rendimiento: Ejercicio 2 vs Ejercicio 3



Modelo 2

OPTIMIZADOR | LR | HIDDEN | OUTPUT

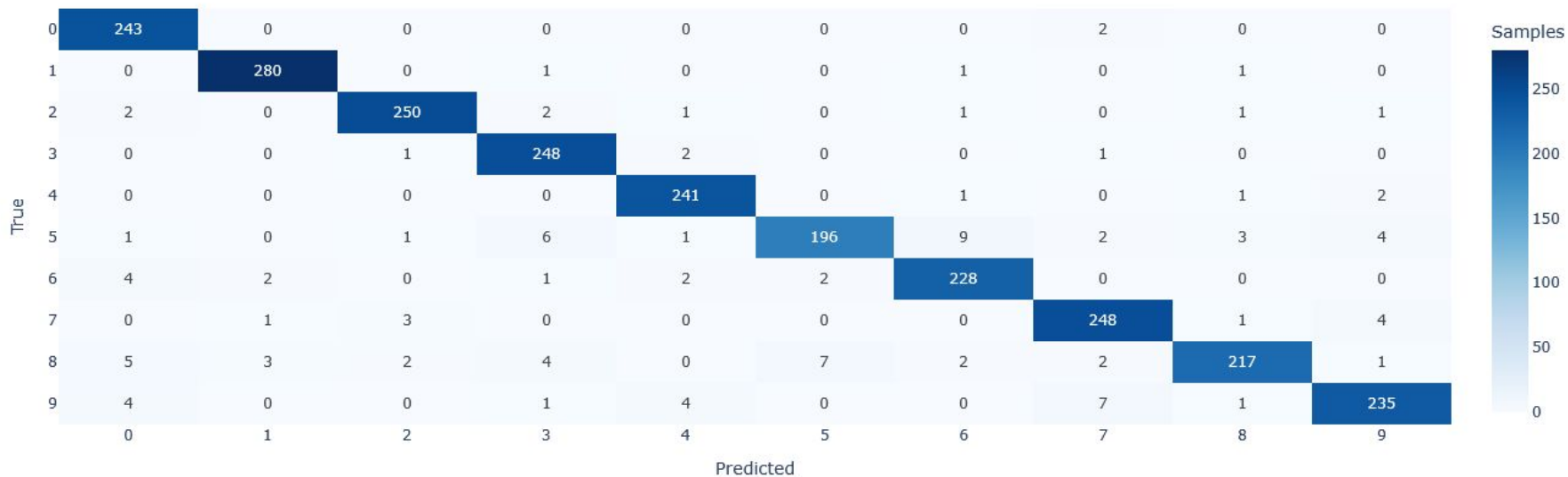
Adam(0.805, 0.932, 3.72e-08) | 0.0019 | ReLU | Sigmoid

Batch: 128

Arquitectura: [784, 512, 256, 128, 64, 10]

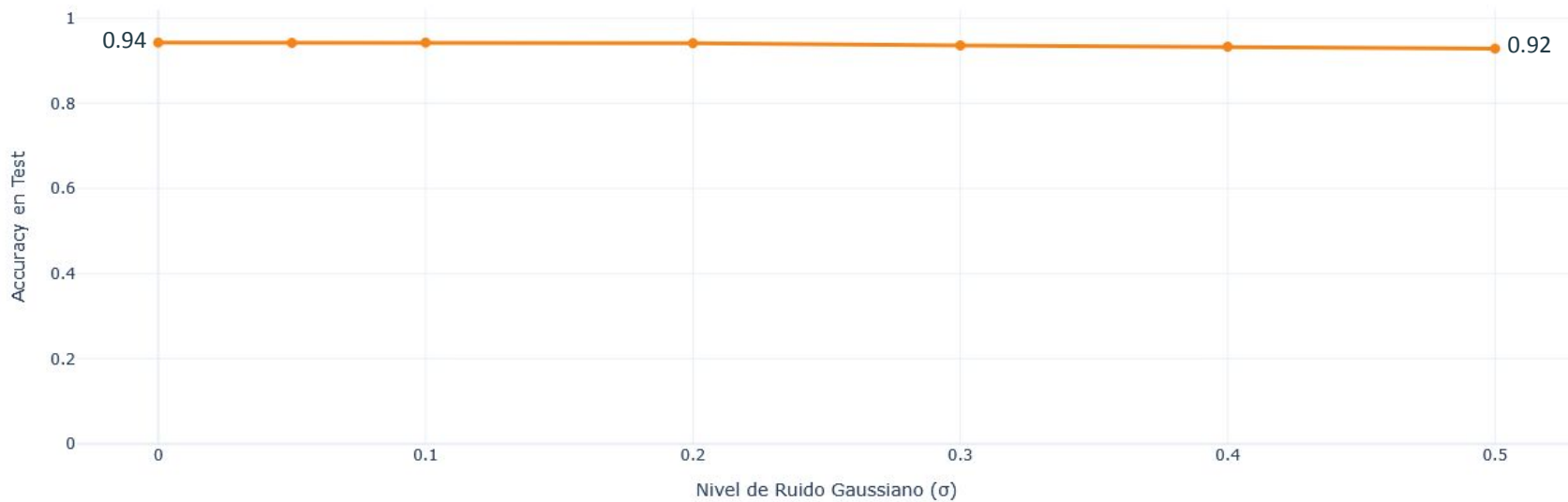
¿Qué mejoró?

Exercise 3 Confusion Matrix (Acc: 0.9555)



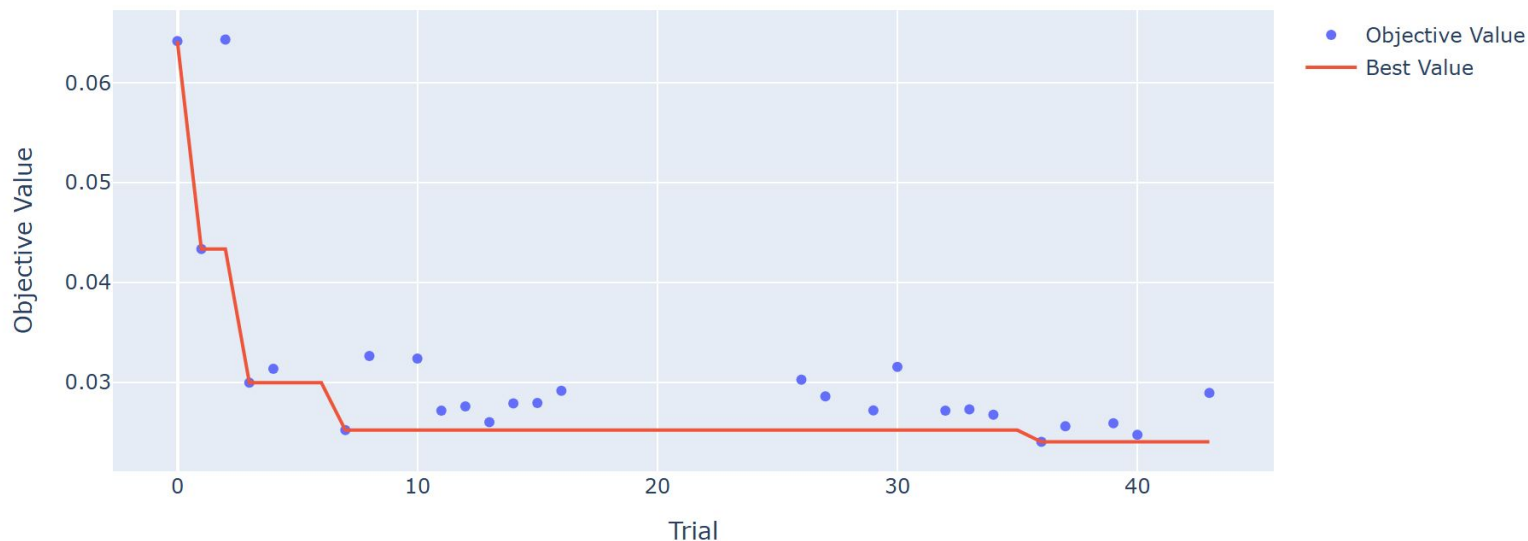
Robustez al ruido

Robustez al Ruido Gaussiano en Test

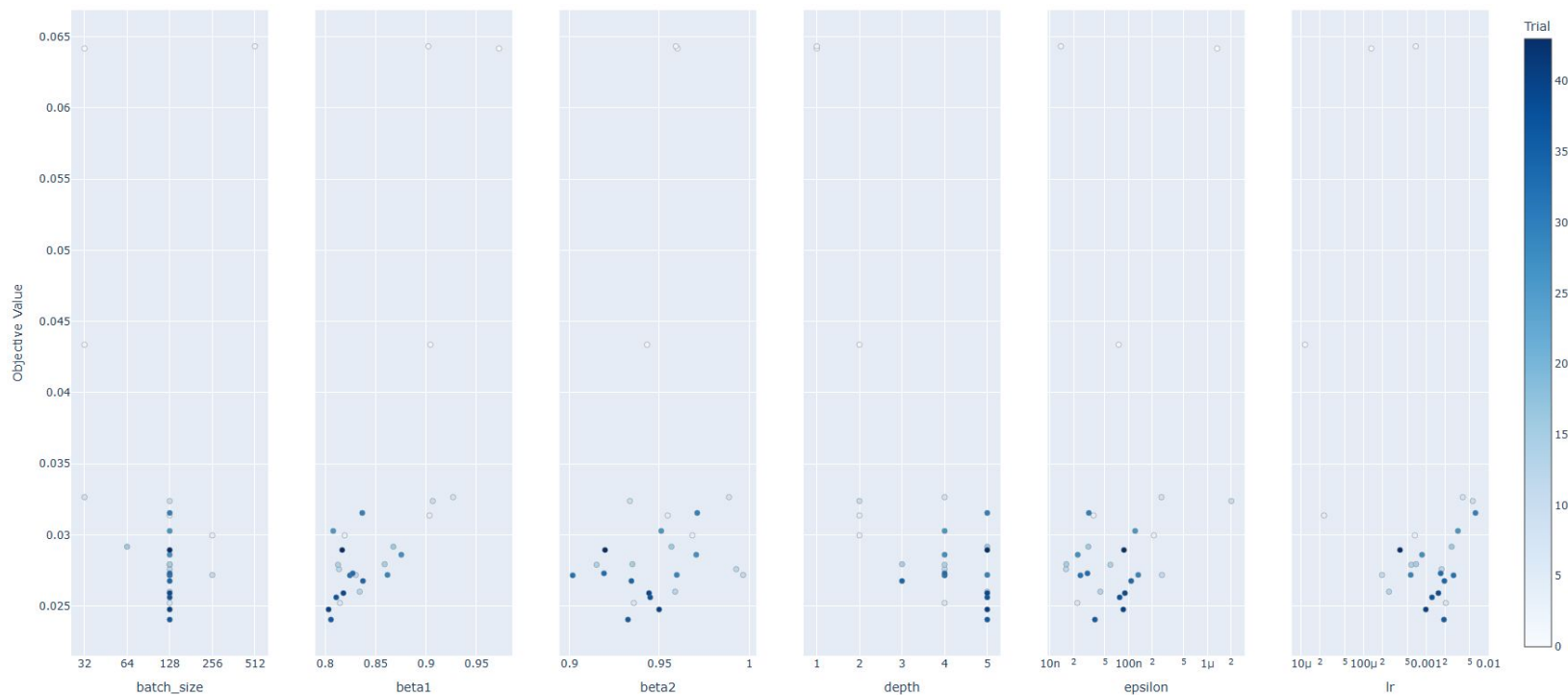


Optimización de hiperparámetros

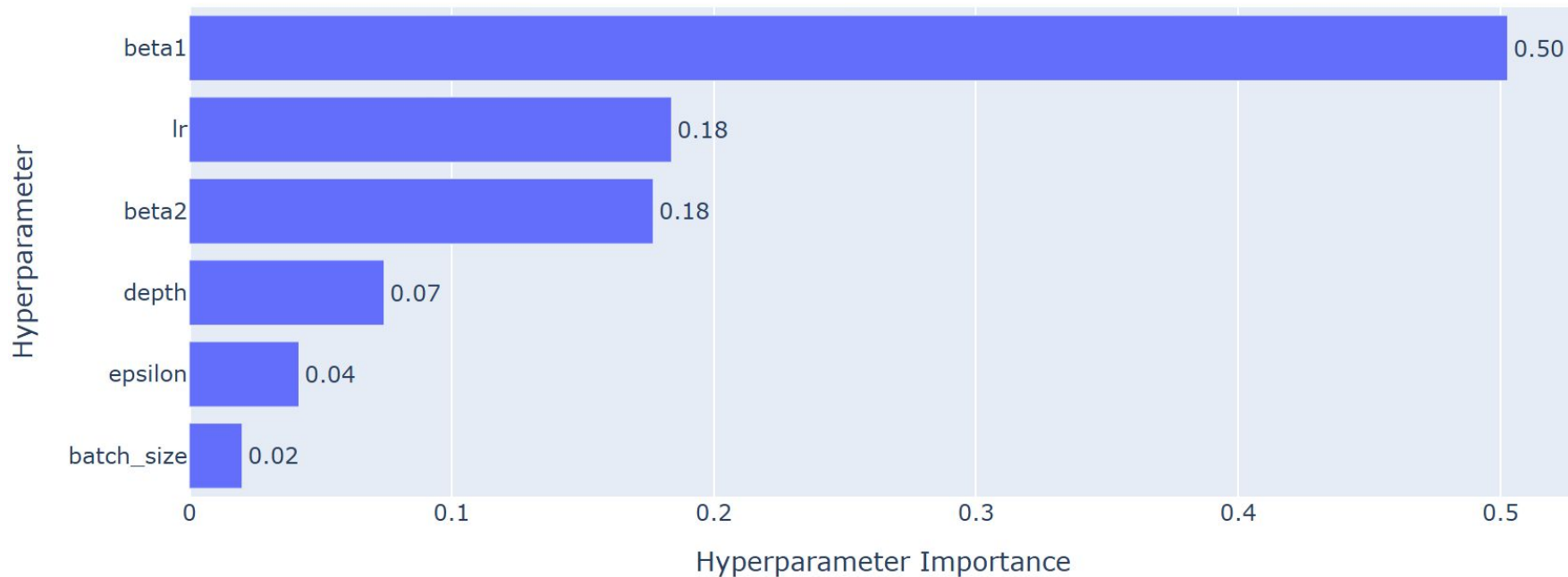
Optimization History Plot

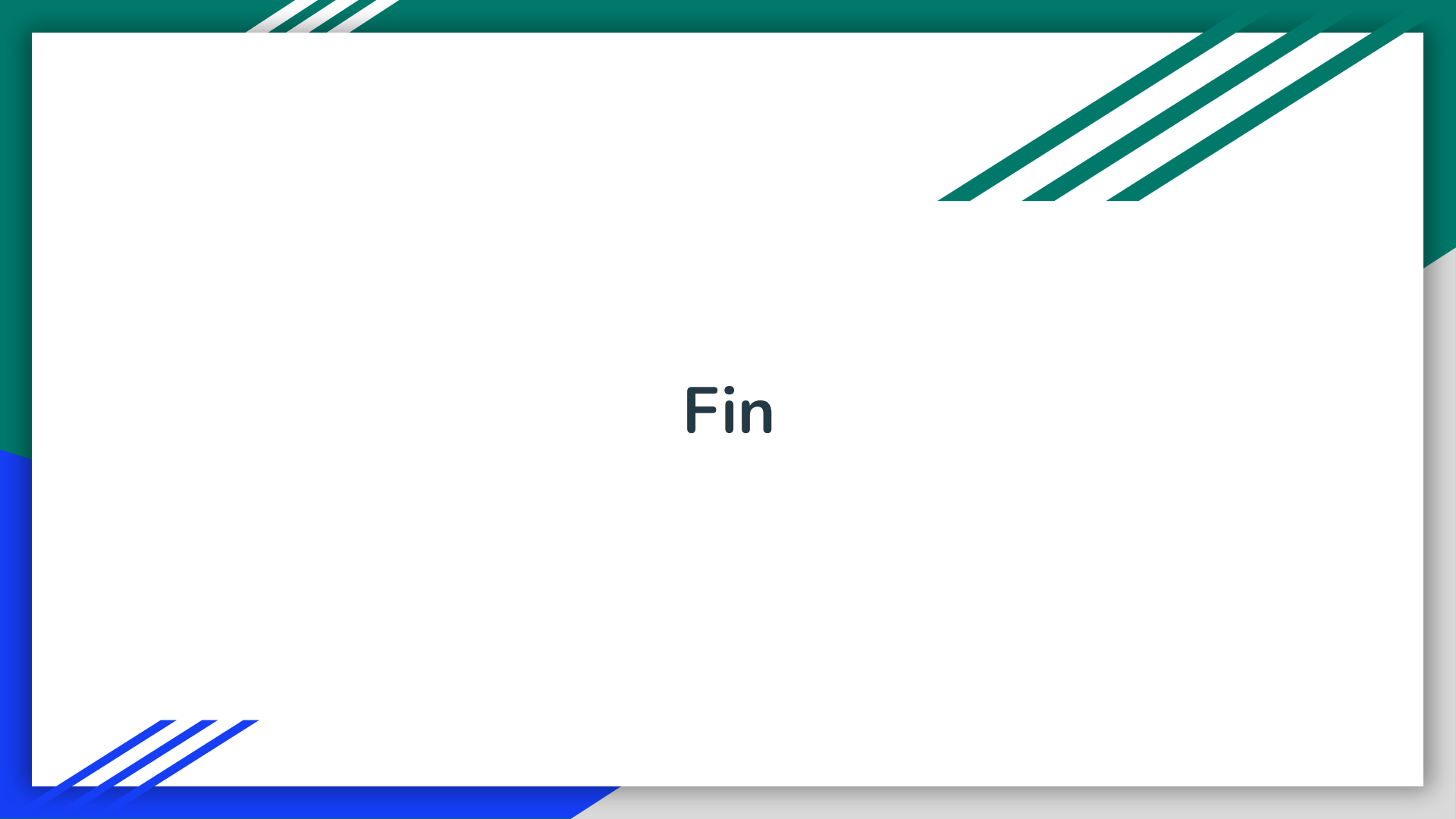


Optimización de hiperparámetros



Optimización de hiperparámetros





Fin



APÉNDICE



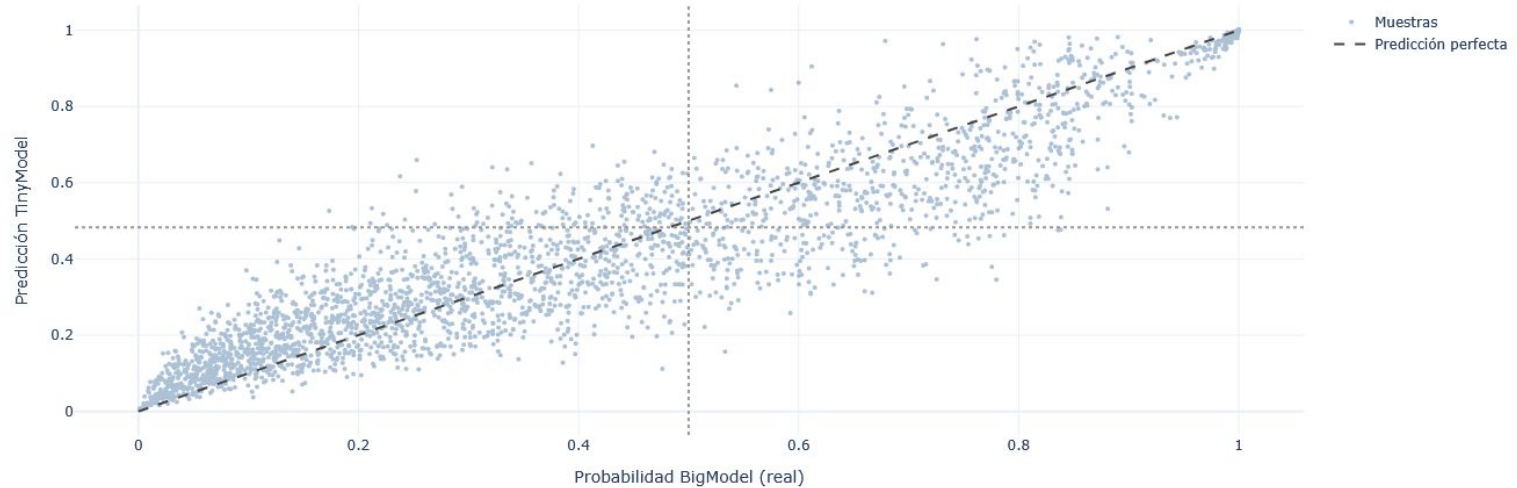
TinyModel

Lineal vs No Lineal - Comparaciones

Loss final:	0.02607	vs	0.01092
MAE:	0.1294	vs	0.0771
RMSE:	0.1614	vs	0.1045
R ² :	0.7151	vs	0.8806
% predicciones fuera de [0, 1]	6%	vs	0.00%

Modelo final y recomendación de umbral.

TinyModel vs BigModel (fuera de muestra; líneas = corte real y umbral)



Modelo final y recomendación de umbral.

Métricas finales (umbral 0.48, corte de fraude 0.5)

Métrica	Global (fuera de muestra)	Media $\pm$ desvío entre folds	IC 95% bootstrap
MAE (prob.)	0.0780	0.0780 $\pm$ 0.0015	—
RMSE (prob.)	0.1051	0.1051 $\pm$ 0.0019	—
R ² (prob.)	0.8792	0.8791 $\pm$ 0.0032	—
Precision	0.8644	0.8644 $\pm$ 0.0138	[0.8520, 0.8778]
Recall (TPR)	0.8496	0.8499 $\pm$ 0.0157	[0.8364, 0.8626]
F1	0.8569	0.8569 $\pm$ 0.0083	[0.8470, 0.8668]
Accuracy	0.8961	0.8961 $\pm$ 0.0064	—
FPR	0.0770	0.0770 $\pm$ 0.0078	—